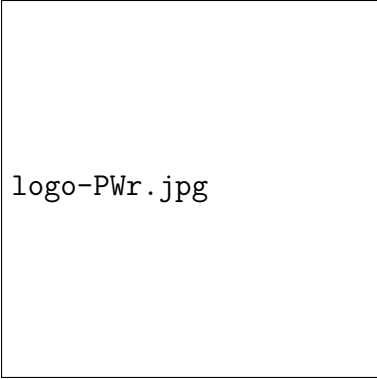




logo-PWr.jpg

Faculty: W4N
Field of study: Computer Engineering, Master's degree

Optimization Methods: Theory and Applications

Subject:
Max-cut problem with evolutionary algorithm and heuristics

Authors:
Wincenty Wensker (272760)
Vadzim Kasyan (294503)
Kostiantyn Slobodnyi (294498)

Date:
14.05.2026r.

Contents

1	Problem Description	2
2	What We Optimize	2
2.1	Objective Function	2
2.2	Practical Evaluation	2
3	Solution Encoding	2
3.1	Encoding Type	2
3.2	Genetic Operators	3
3.3	Symmetry	3
4	Problem Instances	3
4.1	File Format	3
4.2	Instance Families	3
4.3	Summary Table	6
5	Baseline Genetic Algorithm	6
5.1	Algorithm Outline	6
5.2	Selection	7
5.3	Crossover	7
5.4	Mutation	7
5.5	Elitism	8
5.6	Termination Criterion	8
5.7	Configuration Summary	8
5.8	Rationale as a Baseline	9
6	Testing Methodology	9
6.1	Objectives and Scope	9
6.2	Test Matrix	9
6.3	Test Automation	10
6.4	Measured Metrics	10
6.5	Experimental Platform	11
6.6	Reproducibility	12
7	Metaheuristic Enhancements	12
7.1	Greedy-by-Node Repair	12
7.2	Ranked Mutation	13
7.3	Balancer	14
8	Results	15
8.1	Overview	15
8.2	Complete Results Tables	15
8.3	Visualisation	16

9	Analysis	18
9.1	Heuristic effectiveness is instance-structure dependent	18
9.2	Balancer is the most robust single heuristic	18
9.3	Combining heuristics does not compound their benefits	19
9.4	Computational cost is negligible	19

1 Problem Description

The Max-Cut problem is a classical combinatorial optimization problem. Given an undirected graph $G = (V, E)$ with edge weights $w : E \rightarrow \mathbb{R}^+$, the goal is to divide the vertex set V into two disjoint subsets such that the total weight of edges running between them is as large as possible.

The problem is NP-hard, making exact solutions infeasible for large graphs. Practical approaches therefore rely on approximation algorithms or metaheuristics — in this project, an evolutionary algorithm is used.

2 What We Optimize

2.1 Objective Function

Let $S \subseteq V$ denote one side of the partition, with the other side being $V \setminus S$. The quantity being maximized is the total weight of edges whose endpoints lie in different parts of the partition:

$$f(S) = \sum_{\substack{(u,v) \in E \\ u \in S, v \notin S}} w(u, v) \quad (1)$$

The optimal solution is $S^* = \arg \max_{S \subseteq V} f(S)$. For unweighted graphs ($w \equiv 1$) this reduces to maximizing the number of crossing edges.

2.2 Practical Evaluation

Computing $f(S)$ for a given assignment requires a single iteration over all edges, checking whether the two endpoints belong to different parts. This is $O(|E|)$ per evaluation, which remains efficient even for graphs with thousands of edges.

3 Solution Encoding

3.1 Encoding Type

The chosen encoding is binary. Each candidate solution is a vector $x \in \{0, 1\}^{|V|}$ where the i -th bit indicates partition membership of vertex i :

$$x[i] = \begin{cases} 1 & \text{if vertex } i \in S \\ 0 & \text{if vertex } i \notin S \end{cases} \quad (2)$$

This is the natural choice for Max-Cut because the decision space is inherently binary, since every vertex belongs to exactly one of the two partitions. The chromosome length equals $|V|$, and the search space contains $2^{|V|}$ possible solutions (effectively $2^{|V|-1}$ unique cuts due to the symmetry discussed below).

3.2 Genetic Operators

- Mutation: Bit-flip with probability $p_m = 1/|V|$, yielding one expected change per individual per generation.
- Crossover: Single-point or uniform crossover, both directly applicable without any repair step since every binary string represents a valid partition.

3.3 Symmetry

The encoding is symmetric: any solution x and its complement $\bar{x}$ represent the same cut with the same value. To reduce this redundancy, vertex 0 can be fixed to partition S (i.e. $x[0] = 0$ always), halving the effective search space.

4 Problem Instances

All nine instances are drawn from the Biq Mac Library [1], a dedicated collection of Max-Cut and binary quadratic programming benchmarks maintained by Angelika Wiegele at Alpen-Adria-Universität Klagenfurt. The library is the standard reference for testing exact and heuristic Max-Cut solvers, playing an analogous role to TSPLIB for the Travelling Salesman Problem. Instances are grouped into families distinguished by graph topology, edge density, and weight scheme, which together determine the hardness and character of the resulting optimisation landscape.

4.1 File Format

Every instance in the Biq Mac Library is stored in the *rud*y output format, named after the graph generator tool used to produce the instances [2]. The format is a plain-text edge list:

```
n      m
h_1 t_1 c_{h_1,t_1}
h_2 t_2 c_{h_2,t_2}
...
h_m t_m c_{h_m,t_m}
```

The first line contains two integers: n (number of vertices) and m (number of edges). Each of the following m lines describes one undirected edge: h_i and t_i are the one-based indices of the two endpoints, and c_{h_i,t_i} is the integer edge weight. Because each edge is listed exactly once, the full symmetric adjacency is reconstructed during loading by treating every entry as bidirectional.

4.2 Instance Families

g05 — Unweighted Random Graphs

Instances of the form `g05_n.i` are unweighted Erdős–Rényi random graphs with n vertices and edge probability 0.5, generated by *rud*y. All edge weights are 1; the objective reduces

to maximising the number of crossing edges. The uniform structure and moderate density make these good baseline instances: the landscape is relatively smooth, local optima are plentiful but shallow, and the optimal cut value scales predictably with n .

- **g05_60.1** ($n = 60, m \approx 885$) — smallest instance in the suite, used for rapid correctness checks and convergence profiling.
- **g05_80.1** ($n = 80, m \approx 1580$) — same family at larger scale, exposing how the algorithm handles the increased search space 2^{79} while the landscape character remains the same.

pm1s / pm1d — Sparse and Dense Frustrated Graphs

Instances of the form **pm1s_n.i** (sparse, density 0.1) and **pm1d_n.i** (dense, density 0.99) have integer edge weights drawn uniformly from $\{-1, 0, +1\}$. The presence of negative weights introduces *frustration*: satisfying one edge (placing its endpoints in different partitions) forces the violation of a negatively-weighted edge elsewhere. This creates a rugged landscape with exponentially many local optima, making these instances significantly harder than the unweighted g05 family of the same size.

- **pm1s_100.1** ($n = 100, m \approx 990$) — sparse frustrated graph. Low density means the graph is nearly forest-like in places; negative weights cluster around a small fraction of edges, creating isolated pockets of frustration.
- **pm1d_100.9** ($n = 100, m \approx 9900$) — near-complete graph with frustrated weights. The density is the dominant challenge here: almost every pair of vertices is connected, so every partition decision has global consequences. This is the hardest instance among those with $n = 100$.

w / pw — Integer-Weighted Random Graphs

Instances of the form **wd_100.i** have integer weights drawn from $[-10, 10]$, while **pwd_100.i** restricts weights to $[1, 10]$ (positive only). The wider weight range relative to the **pm1** family means individual edges differ substantially in importance, so the objective landscape is more varied and greedy decisions based on single-edge gain are more informative.

- **w01_100.1** ($n = 100$, density 0.1, weights $[-10, 10]$) — sparse graph with signed wide-range weights. The combination of low density and large negative weights produces deep, narrow local optima.
- **w09_100.9** ($n = 100$, density 0.9, weights $[-10, 10]$) — dense counterpart. Near-complete connectivity with wide signed weights; the benchmark role here is analogous to **pm1d** but with a richer weight distribution that stresses weight-aware operators.
- **pw09_100.9** ($n = 100$, density 0.9, weights $[1, 10]$) — positive-only wide weights at high density. Because no negative weights are present, frustration is absent and the landscape is smoother than **w09**. The contrast between this instance and **w09_100.9** isolates the effect of negative weights on algorithm performance.

t2g — Two-Dimensional Toroidal Ising Spin Glasses

Instances of the form **t2gn_seed** are two-dimensional toroidal grid graphs of size $\sqrt{n} \times \sqrt{n}$ (so n is a perfect square) with Gaussian-distributed real-valued edge weights, generated following the statistical physics model of Frauke Liers [3]. Each vertex is connected to its four grid neighbours (up, down, left, right) with wrap-around boundary conditions, giving exactly $m = 2n$ edges. The Gaussian weights can be positive or negative, yielding frustrated cycles that are the defining feature of the *Ising spin glass* model. The highly regular topology combined with frustration makes these instances notoriously difficult for local search: the energy landscape has exponentially many near-degenerate local optima and the graph diameter is small, so perturbations propagate quickly.

- **t2g10_5555** ($n = 100$, $m = 200$, seed 5555) — 10×10 toroidal Ising instance. Despite having only 200 edges, this instance is harder than any of the **g05** or **pm1s** instances at $n = 100$ because of the structured frustration inherent to the spin-glass topology. Used to evaluate how the algorithm behaves on highly structured graphs with a rugged, correlated landscape.
- **t2g10_6666** ($n = 100$, $m = 200$, seed 6666) — identical topology and weight distribution, different random seed. Including two instances from the same family with different seeds allows assessment of landscape sensitivity: if algorithm performance varies substantially between **t2g10_5555** and **t2g10_6666**, the heuristic is sensitive to the specific realisation of the disorder rather than robustly solving the spin-glass structure.

4.3 Summary Table

Instance	Family	$ V $	$ E $	Weights	Key trait
g05_60.1	g05	60	≈ 885	$\{0, 1\}$	Unweighted, baseline
g05_80.1	g05	80	≈ 1580	$\{0, 1\}$	Unweighted, scaling
pm1s_100.1	pm1s	100	≈ 990	$\{-1, 0, 1\}$	Sparse, frustrated
pm1d_100.9	pm1d	100	≈ 9900	$\{-1, 0, 1\}$	Dense, heavy frustration
w01_100.1	w	100	≈ 990	$[-10, 10]$	Sparse, wide signed weights
w09_100.9	w	100	≈ 9900	$[-10, 10]$	Dense, wide signed weights
pw09_100.9	pw	100	≈ 9900	$[1, 10]$	Dense, positive-only weights
t2g10_5555	t2g	100	200	Gaussian	2D torus Ising spin-glass
t2g10_6666	t2g	100	200	Gaussian	Same topology, different seed

Table 1: Summary of the nine benchmark instances from the Biq Mac Library. The three instances intended for primary benchmarking are **pm1d_100.9**, **w09_100.9**, and **pw09_100.9**, as their near-complete graphs represent the most demanding workload in the collection.

5 Baseline Genetic Algorithm

Before introducing problem-specific heuristics it is necessary to establish a reference implementation: a genetic algorithm that uses only standard, well-studied components with no domain knowledge built in. All performance comparisons in this project measure improvement relative to this baseline, so its design choices must be well-justified and representative of mainstream practice.

5.1 Algorithm Outline

The algorithm follows the canonical generational GA loop. An initial population of μ individuals is created by assigning each gene independently and uniformly at random, with vertex 0 fixed to partition S throughout (see Section 3). Each individual is evaluated once immediately after creation. The loop then repeats until the fitness function evaluation budget is exhausted: a new population of size μ is assembled from the current one, the current

population is discarded, and the new population takes its place. The best individual ever seen across all generations is tracked separately and returned as the final answer.

5.2 Selection

Parent selection uses *tournament selection* with tournament size $k = 3$. Three individuals are drawn uniformly at random from the population (with replacement) and the one with the highest fitness wins the tournament and is used as a parent. The process is repeated independently to obtain the second parent.

Tournament selection was chosen over fitness-proportionate (roulette-wheel) selection for two reasons. First, it is robust to fitness scaling issues: it depends only on the relative ordering of fitness values within each tournament, not on their absolute magnitudes or ratios. This is important for Max-Cut instances where the range of cut values can vary by orders of magnitude across instance families. Second, the selection pressure is easily controlled by the tournament size k : larger k increases pressure (the best individual wins more often), smaller k reduces it. The value $k = 3$ is the most common default in the literature, providing moderate pressure without converging prematurely.

5.3 Crossover

Offspring are produced by *uniform crossover* applied with probability $p_c = 0.8$. Each gene of the offspring is inherited from parent A or parent B with equal probability, independently for every position. If crossover is not applied (probability $1 - p_c = 0.2$), the offspring is an unmodified copy of parent A.

Uniform crossover was preferred over single-point or two-point crossover because it does not impose any positional bias: there is no notion of a “building block” region defined by chromosomal proximity, which is particularly appropriate here since the vertex indices are arbitrary labels with no inherent spatial relationship. For Max-Cut on general graphs, vertices that are functionally related (connected by heavy edges) are not necessarily adjacent in the chromosome, so crossover operators that respect positional locality offer no structural benefit and may actively fragment meaningful partial solutions. The crossover rate of 0.8 is the standard default used in the majority of GA studies and represents a strong but not exclusive preference for recombination over cloning.

5.4 Mutation

After crossover, every gene in the offspring is independently subjected to *bit-flip mutation*: the gene is inverted with probability

$$p_m = \frac{1}{|V|} \tag{3}$$

This yields an expected number of exactly one flip per offspring per generation, regardless of chromosome length. The $1/|V|$ rule is the canonical default for binary GAs and has a solid theoretical basis: with smaller rates the algorithm explores too slowly, while with larger

rates the search degrades toward random walk. Vertex 0 is never flipped, consistent with the symmetry convention.

5.5 Elitism

The two fittest individuals from the current generation are copied directly into the next population without modification (*elitism count* = 2). The remaining $\mu - 2$ slots are filled by offspring from crossover and mutation. Elitism guarantees that the best solution found so far is never lost to random variation, which stabilises convergence and prevents regression. Two elites rather than one are retained to preserve a small amount of genetic diversity at the top of the fitness ranking while still providing strong protection against fitness loss.

5.6 Termination Criterion

The algorithm terminates when the total number of fitness function evaluations reaches a fixed *budget* B (default $B = 50,000$). The budget is enforced at the level of individual offspring: generation boundaries are not synchronisation points, and the loop exits as soon as the counter reaches B even if the current generation is not complete.

A fixed evaluation budget was chosen as the termination criterion in preference to a fixed generation count because it provides a fair basis for comparison between algorithms with different population sizes or offspring counts. Two algorithms that both consume 50,000 evaluations have spent the same computational effort on the objective function, regardless of how they internally structured that effort into generations. This is the standard approach in algorithm benchmarking studies, in particular on the BBOB/COCO platform. A fixed generation count, by contrast, conflates the number of generations with the cost per generation, making cross-algorithm comparisons ambiguous.

5.7 Configuration Summary

Parameter	Value
Population size μ	100
NFE budget B	50,000
Selection	Tournament, $k = 3$
Crossover	Uniform, $p_c = 0.8$
Mutation	Bit-flip, $p_m = 1/ V $
Elitism	2 individuals
Initialisation	Uniform random (vertex 0 fixed to 0)

Table 2: Parameter configuration of the baseline genetic algorithm. All three metaheuristic enhancements are disabled in this configuration.

5.8 Rationale as a Baseline

The configuration above was deliberately constructed to be *generic*: no parameter depends on the structure of the graph, the weight distribution, or any property of the specific instance being solved. Every component (tournament selection, uniform crossover, $1/|V|$ bit-flip mutation, two-elite replacement) is a well-established default that appears in textbook treatments of genetic algorithms [5, 6] and is representative of what a practitioner would implement without any domain knowledge.

This genericity is precisely what makes it a useful baseline. Any improvement demonstrated by the metaheuristic enhancements described in Section 7 is unambiguously attributable to the exploitation of problem structure, since the underlying algorithm, the evaluation budget, and all other parameters remain identical. A more sophisticated baseline, for instance one using adaptive parameter control or a problem-specific repair operator by default, would make it harder to isolate the contribution of each individual enhancement.

The choice of $\mu = 100$ and $B = 50,000$ yields approximately 500 full generations for the default configuration, which is sufficient for convergence on the benchmark instances used while keeping wall-clock time well under one second per run. This allows large numbers of repeated trials to be conducted efficiently, which is important given the stochastic nature of the algorithm.

6 Metaheuristic Enhancements

The baseline evolutionary algorithm described in Section 5 operates with a uniform crossover, a flat bit-flip mutation, and purely random initialisation. While this constitutes a sound and reproducible reference point, the algorithm makes no use of problem structure: every vertex is treated identically regardless of how influential it is in the graph, and every offspring is allowed to deviate arbitrarily from any notion of a well-formed partition. The three enhancements described in this section each address a distinct weakness of the baseline and are designed to compose with one another without conflict. All three are optional and controlled by configuration parameters, so the baseline behaviour is recovered exactly when all enhancements are disabled.

A central concept shared by two of the three mechanisms is a *node ranking*. Before the evolutionary loop begins, every vertex $v \in V$ is assigned a scalar importance score equal to the sum of absolute edge weights over all its incident edges:

$$\sigma(v) = \sum_{(v,u) \in E} |w(v,u)| \quad (4)$$

Vertices are then sorted in descending order of σ , producing a static ranking $v_{(1)}, v_{(2)}, \dots, v_{(|V|)}$ where $v_{(1)}$ is the most influential vertex and $v_{(|V|)}$ the least. This ranking is computed once at startup and re-used throughout the run, so its cost is negligible. Its purpose is to provide a principled, domain-aware notion of vertex importance that the heuristic operators can exploit.

6.1 Greedy-by-Node Repair

The greedy-by-node repair mechanism addresses the fact that the baseline crossover and mutation operators are oblivious to the structure of the cut. After crossing two parents, the resulting offspring may assign influential, heavily connected vertices to whichever partition happened to be inherited, with no regard for whether that assignment is locally sensible.

The mechanism works in two phases. In the first phase, carried out once before the evolutionary loop, a greedy reference solution is constructed using the node ranking. Vertices are processed in order of decreasing importance $\sigma(v)$. For each vertex, the cut contributions of placing it in partition S or $V \setminus S$ are evaluated with respect to all already-assigned neighbours: the vertex is assigned to whichever side maximises the weight of crossing edges to those neighbours. If no neighbour has been assigned yet, the vertex receives a random assignment. Vertex 0 is fixed to S throughout, consistent with the symmetry convention described in Section 3. The result is a single greedy partition x^g that respects the importance ordering and serves as a structural reference.

In the second phase, during offspring generation, a fraction r of offspring (configurable, default 0.5) are subjected to repair after crossover and before mutation. Repair overwrites the genes of the top $\lceil \alpha \cdot |V| \rceil$ vertices in the ranking (default $\alpha = 0.2$, i.e. the top 20 %) with the values from x^g , leaving the remaining genes untouched. Mutation is then applied to the repaired offspring, allowing the lower-ranked genes to continue evolving freely while the core partition structure imposed by the greedy solution is preserved.

The rationale is that the most influential vertices dominate the cut value. Fixing their assignments to a greedy optimum reduces the effective search space to the lower-ranked, less critical vertices, where the evolutionary operators have the greatest relative freedom and where random variation causes the least structural damage. This is a form of problem decomposition guided by domain knowledge rather than random sampling.

6.2 Ranked Mutation

The standard bit-flip mutation applies the same probability $p_m = 1/|V|$ to every vertex. This is appropriate when all vertices are equally important, but in Max-Cut the importance distribution is highly skewed: a small number of high-degree, high-weight vertices account for a disproportionate share of the objective value, while the majority of vertices have limited influence. Applying the same mutation pressure to a hub vertex as to a peripheral leaf is wasteful at best and destructive at worst.

The ranked mutation operator replaces the uniform rate with a per-vertex rate derived from the node ranking. The rate for vertex v at rank k (where rank 1 is most important) is computed by linear interpolation between a minimum multiplier $\mu_{\min}$ and a maximum multiplier $\mu_{\max}$:

$$p_m(v_{(k)}) = p_m^{\text{base}} \cdot \left(\mu_{\min} + \frac{k-1}{|V|-1} (\mu_{\max} - \mu_{\min}) \right) \quad (5)$$

The top-ranked vertex receives rate $p_m^{\text{base}} \cdot \mu_{\min}$ and the bottom-ranked vertex receives $p_m^{\text{base}} \cdot \mu_{\max}$. With default values $\mu_{\min} = 0.25$ and $\mu_{\max} = 2.0$, the most influential vertex is

mutated at one quarter of the base rate and the least influential vertex at twice the base rate, an eight-fold spread across the ranking.

Optionally, the per-vertex rates can be normalised so that their mean equals p_m^{base} , preserving the expected number of flips per offspring at approximately one. Normalisation ensures that the ranked and flat mutation operators consume the same average mutation budget, making any performance difference attributable purely to the redistribution of mutation pressure rather than to a change in overall mutation intensity. When normalisation is disabled, the operator instead applies strictly more mutation pressure to low-ranked vertices and less to high-ranked ones, at the cost of making the comparison with the baseline less direct.

The mechanism is complementary to the greedy-by-node repair: repair fixes the top vertices to a greedy reference, while ranked mutation suppresses random perturbation of those same vertices and concentrates exploratory variation on the periphery of the ranking where it is least likely to damage the core partition structure.

6.3 Balancer

Neither the baseline operators nor the two mechanisms above impose any constraint on the size of the two partitions. Crossover and mutation can freely produce heavily asymmetric solutions in which one side of the partition contains the vast majority of vertices. Such solutions are not invalid — every binary string represents a legal partition — but they are structurally poor: a partition with $|S| \approx 1$ and $|V \setminus S| \approx |V|$ can cut at most a small fraction of edges by volume, and the evolutionary dynamics that produced it are unlikely to escape toward more balanced, higher-quality regions without a structural correction.

The balancer operator corrects excessive imbalance after mutation. It is parameterised by a target interval $[\beta_{\min}, \beta_{\max}]$ for the fraction of vertices assigned to S (default $[0.4, 0.6]$, i.e. the 40%–60% band). After mutation, if $|S|/|V|$ falls within the target interval the operator does nothing. If the partition is outside the interval, the operator identifies the larger side and moves vertices from that side to the other until balance is restored.

The choice of which vertices to flip is not arbitrary. For each vertex v on the larger side, the operator computes the *local flip gain*:

$$g(v) = \sum_{(v,u) \in E} w(v,u) \cdot [x[u] \neq x[v]] - \sum_{(v,u) \in E} w(v,u) \cdot [x[u] = x[v]] \quad (6)$$

That is, $g(v)$ is the net change in cut value that would result from moving v to the other partition, considering only edges incident to v . The operator sorts the candidate vertices by $g(v)$ in descending order and flips from the bottom of the list: the vertices with the smallest (or most negative) gain are those contributing least to the cut in their current position and are therefore the least costly to move. Vertex 0 is never flipped, preserving the symmetry convention.

This greedy selection strategy ensures that the rebalancing step causes as little damage to the objective as possible. Unlike the greedy-by-node repair, which enforces a specific partition structure, the balancer makes only the minimum number of targeted changes needed to restore dimensional feasibility, and does so in a locally optimal order. Fitness is evaluated

once after all three operators (repair, ranked mutation, balancer) have been applied, so the balancer incurs no additional objective function calls.

7 Testing Methodology

7.1 Objectives and Scope

The purpose of the experimental evaluation is to compare the performance of the baseline genetic algorithm (Section 5) against each of the three metaheuristic enhancements (Section 7), both individually and in combination. Because the algorithm is stochastic, a single run carries no statistical meaning; repeated trials are required to obtain reliable estimates of central tendency and variance for each configuration.

The evaluation is structured around two questions. First, does each individual enhancement improve upon the baseline on the instance families for which it was designed? Second, does combining all three enhancements simultaneously yield further improvement over any single enhancement, or does the interaction between them introduce interference that reduces the benefit of each component?

7.2 Test Matrix

Every combination of instance and configuration is treated as an independent experiment. The full test matrix is as follows:

- **Standard experiments:** all nine benchmark instances crossed with four configurations — baseline, greedy-by-node only, ranked mutation only, and balancer only — each repeated **5 times**. This yields $9 \times 4 \times 5 = 180$ runs and allows the isolated effect of each heuristic to be measured across the full range of instance families.
- **Primary experiments:** the three hardest instances (`pm1d_100.9`, `w09_100.9`, `pw09_100.9`) with all three heuristics enabled simultaneously, each repeated **10 times**. This yields $3 \times 10 = 30$ runs and provides a higher-confidence estimate of the combined configuration on the instances where improvement is expected to be most visible.

The total number of runs is 210. All five configurations for the primary instances are therefore available for comparison: baseline, each single heuristic at 5 runs, and the combined configuration at 10 runs.

7.3 Test Automation

All experiments are executed by a dedicated Python script (`run_tests.py`) that drives the compiled C++ binary (`ga_main`) as a subprocess. The binary is invoked with the `--json` flag, which suppresses the human-readable console output and instead emits a single JSON object per run on standard output. This eliminates any text-parsing ambiguity and allows the runner to ingest results directly via the standard library JSON parser with no post-processing.

For each run the script records: the instance file, the configuration file, the active heuristic flags, the best cut value found, the total number of fitness function evaluations consumed, the number of complete generations executed, the wall-clock time of the evolutionary loop in milliseconds, the stop reason, and the full best-solution vector. After all runs complete, the script writes three output files:

- **results/raw.jsonl** — one JSON record per run in newline-delimited format, containing all of the above fields including the complete solution vector. This file is the primary archival record and can be loaded directly into data-analysis tools such as pandas.
- **results/summary.csv** — one row per (instance, configuration) pair with aggregate statistics (best, worst, mean, standard deviation of cut value; mean and standard deviation of wall-clock time; mean NFE and generation count).
- **results/report.txt** — a human-readable formatted table identical in content to the CSV but intended for direct reading.

7.4 Measured Metrics

The following quantities are recorded for every run and used in the analysis.

Best cut value. The primary performance metric is $f^* = \max_t f(x_t)$, the highest cut value found by the algorithm across all generations in a single run. This is the quantity the algorithm directly optimises and is the natural basis for comparing solution quality across configurations.

Mean and standard deviation of cut value. Aggregated over repeated runs, the mean $\bar{f}^*$ measures the expected solution quality and the standard deviation σ_{f^*} measures consistency. A configuration with a high mean but also a high standard deviation performs well on average but is unreliable; a configuration with a lower mean but near-zero standard deviation is more predictable. Both quantities are needed to assess practical usefulness.

Number of fitness function evaluations (NFE). All configurations share the same budget $B = 50,000$ evaluations and the termination criterion is the budget exhaustion (see Section 5). The NFE reported per run is therefore always equal to B under normal operation, and its inclusion in the output primarily serves as a consistency check. In future experiments with early-stopping criteria, the NFE would become a meaningful variable quantity.

Wall-clock time. The elapsed time of the evolutionary loop is measured in milliseconds using a high-resolution monotonic clock. It is reported per run and aggregated as mean and standard deviation. Wall-clock time reflects the combined cost of fitness evaluations, selection, crossover, mutation, and any heuristic repair operations. Because the NFE budget is fixed, differences in wall-clock time between configurations directly quantify the computational overhead introduced by each metaheuristic enhancement.

Number of generations. The number of complete generations executed within the NFE budget depends on the population size, the elitism count, and whether any per-generation offspring are skipped when the budget is exhausted mid-generation. It is recorded primarily to confirm that all configurations exercise a comparable number of evolutionary cycles.

7.5 Experimental Platform

All experiments were conducted on a single machine with no other user workloads active during the test runs. Effort was made to minimise interference from background operating system processes: no interactive applications were running during the experiments and the machine had been in continuous operation for several days prior, ensuring that any OS caching and memory management effects had stabilised.

Property	Value
Model	Apple Macbook Pro 14"
Operating system	macOS 26.5.1 (arm64)
Kernel	Darwin 25.5.0
CPU	Apple M4 Pro
Memory	24,576 MiB (24 GB unified)
Compiler	g++ with <code>-std=c++17 -O2</code>
Python	CPython 3.14 (test runner only)

Table 3: Hardware and software configuration of the experimental platform.

The Apple M4 Pro is an ARM-based system-on-chip with a unified memory architecture. The binary was compiled natively for arm64 with full optimisation (`-O2`) and no architecture-specific tuning flags beyond the toolchain defaults. Because the algorithm is single-threaded, the number of performance cores available is not relevant; the results reflect single-core throughput only. The 210 runs of the full test matrix completed in approximately 22 seconds of wall-clock time, with individual runs averaging 65–80 ms each depending on the instance and configuration.

7.6 Reproducibility

The random seed is set via the operating system random device (`/dev/urandom`) independently for each run, meaning that repeated invocations of the test script will produce different specific solutions but statistically equivalent distributions. To reproduce exact numerical results, a fixed seed can be set via the `random_seed` parameter in the configuration file. The full solution vector for every run is stored in `results/raw.jsonl`, which allows any individual result to be inspected or re-evaluated offline.

8 Results

8.1 Overview

This section presents the numerical results of the 210-run benchmark described in Section 6. All runs used the same NFE budget of $B = 50,000$ and the parameter defaults from Table 2. Results are organised into two parts: a complete numerical tables covering all nine instances and all configurations, followed by three figures that visualise the patterns across instance families.

8.2 Complete Results Tables

Tables 4–7 report, for every (instance, configuration) pair, the best cut value found across all runs, the mean and standard deviation of the cut value, and the mean wall-clock time of the evolutionary loop. Configurations are listed in the order: baseline, greedy-by-node, ranked mutation, balancer, and (for primary instances) all three combined.

Instance	Config	Best	Mean	Std	Time (ms)
g05_60.1	Baseline	528	524.4	3.3	54.3
	Greedy	530	526.4	2.5	54.9
	Ranked Mut.	531	526.0	4.4	54.1
	Balancer	532	528.4	5.0	54.3
g05_80.1	Baseline	941	933.8	8.9	76.5
	Greedy	922	914.6	5.0	77.5
	Ranked Mut.	941	931.6	14.2	77.7
	Balancer	941	926.8	13.2	76.7

Table 4: Results for the g05 family (unweighted random graphs). All values are over 5 runs.

Instance	Config	Best	Mean	Std	Time (ms)
pm1s_100.1	Baseline	125	117.0	5.3	69.5
	Greedy	121	115.8	3.0	70.1
	Ranked Mut.	123	120.8	2.2	69.2
	Balancer	126	119.2	5.4	69.6
pm1d_100.9	Baseline	405	372.2	18.5	145.7
	Greedy	399	383.6	12.9	146.5
	Ranked Mut.	396	372.6	17.4	145.9
	Balancer	390	372.6	12.8	146.9
	All Three	391	366.2	16.2	146.8

Table 5: Results for the pm1 family (frustrated $\{-1, 0, +1\}$ -weighted graphs). pm1d also includes 10 runs of the combined configuration.

Instance	Config	Best	Mean	Std	Time (ms)
w01_100.1	Baseline	719	671.8	33.4	70.3
	Greedy	717	676.6	26.6	70.1
	Ranked Mut.	688	663.6	20.9	70.0
	Balancer	711	681.4	34.2	71.4
w09_100.9	Baseline	1983	1890.8	70.5	138.5
	Greedy	1941	1900.8	35.6	139.3
	Ranked Mut.	1989	1870.2	69.5	138.3
	Balancer	1980	1891.6	79.3	142.0
	All Three	1971	1884.7	55.8	139.7
pw09_100.9	Baseline	13537	13506.4	22.0	139.6
	Greedy	13605	13549.8	49.3	138.6
	Ranked Mut.	13645	13586.0	51.7	137.9
	Balancer	13638	13589.6	56.2	137.6
	All Three	13553	13508.7	43.6	139.7

Table 6: Results for the **w** and **pw** families (wide-range integer-weighted graphs).

Instance	Config	Best	Mean	Std	Time (ms)
t2g10.5555	Baseline	5,805,278	5,640,740	129,520	64.6
	Greedy	5,781,807	5,596,272	187,058	65.5
	Ranked Mut.	5,703,835	5,440,968	264,285	65.0
	Balancer	5,716,790	5,592,813	124,511	65.0
t2g10.6666	Baseline	5,371,427	5,089,528	179,088	64.6
	Greedy	5,373,699	5,295,447	91,005	65.3
	Ranked Mut.	5,421,665	5,130,520	198,295	64.4
	Balancer	5,457,543	5,252,944	179,215	66.0

Table 7: Results for the **t2g** family (2D toroidal Ising spin-glass instances). All values are over 5 runs.

8.3 Visualisation

Figure 1 shows the relative change in mean cut value compared to the baseline for each heuristic across all nine instances. Figure 2 focuses on the three primary instances with error bars and includes the combined configuration. Figure 3 confirms that the heuristics impose negligible wall-clock overhead.

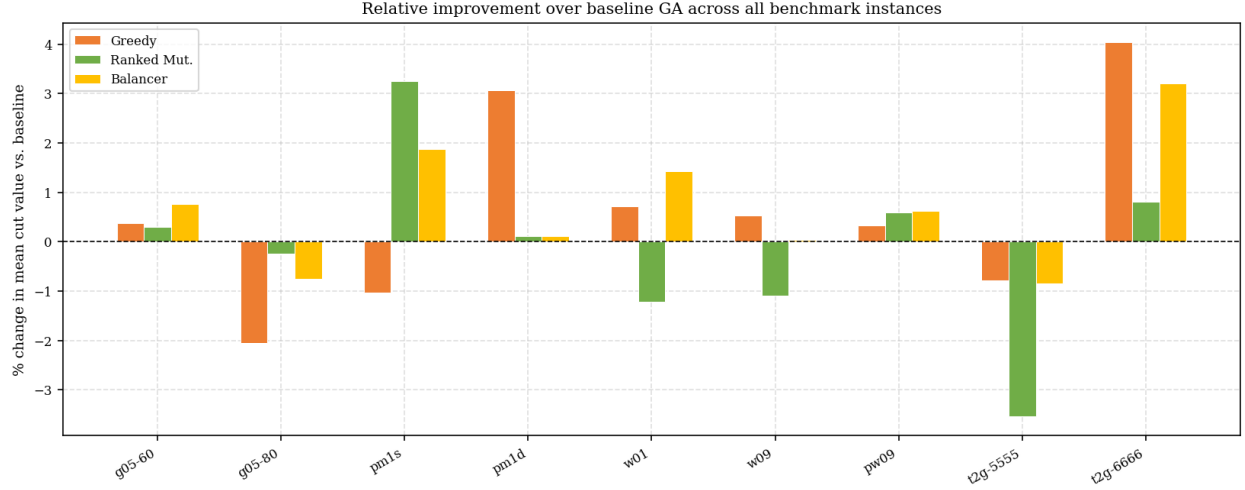


Figure 1: Relative change in mean cut value compared to the baseline GA (no heuristics), across all nine benchmark instances. Positive values indicate improvement over the baseline. Each bar is the mean over five independent runs.

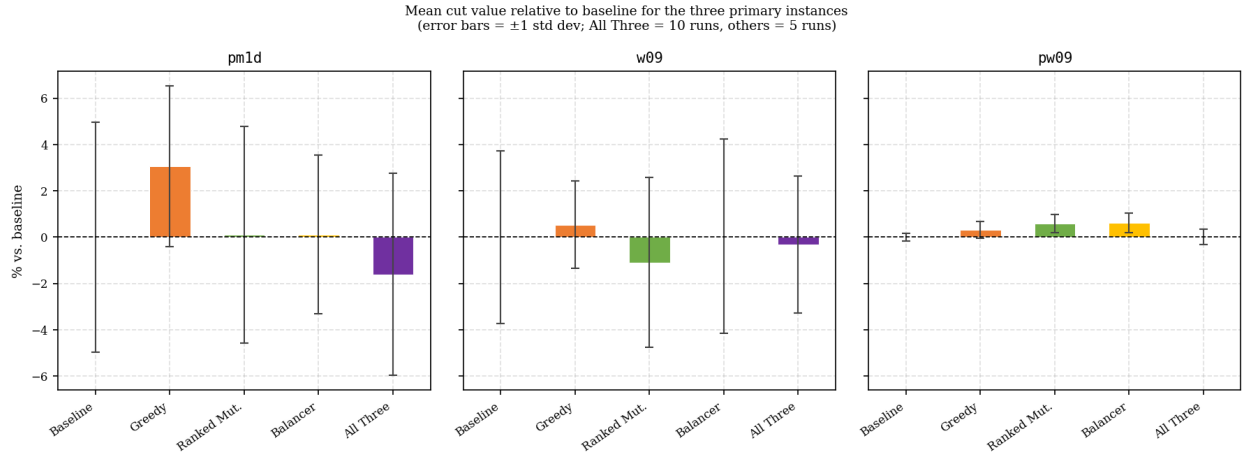


Figure 2: Mean cut value relative to the baseline (%) for the three primary benchmark instances (pm1d_100.9, w09_100.9, pw09_100.9). Error bars show ± 1 standard deviation over the repeated runs. The dashed line marks zero (baseline level). The *All Three* bar corresponds to 10 runs; all others to 5.



Figure 3: Wall-clock execution time of the evolutionary loop relative to the baseline (%), per instance and heuristic. All deviations are within $\pm 3\%$, confirming that the heuristic operators introduce negligible computational overhead.

9 Analysis

9.1 Heuristic effectiveness is instance-structure dependent

The clearest pattern in the results (Figure 1) is that the benefit of each heuristic correlates strongly with the structural properties of the instance rather than its size. The greedy-by-node repair achieves its largest gains on the two instance families with the most heterogeneous weight distributions: **pm1d** and **t2g** (Tables 5 and 7), where the node ranking reliably identifies a small influential core. On the unweighted **g05** family, where all edge weights are equal and the ranking carries little information, the same heuristic is neutral or slightly harmful (Table 4). This suggests a general principle: repair mechanisms that fix a subset of vertices are only as useful as the ranking that selects them, and weight heterogeneity is the primary driver of ranking quality.

9.2 Balancer is the most robust single heuristic

Across all nine instances the balancer produces no degradation larger than 1% and delivers the best or near-best single-heuristic result on five of them (Figure 1). Unlike the greedy repair and ranked mutation — both of which depend on the informativeness of the node ranking — the balancer acts on a structural property of the partition that is instance-agnostic: excessive size asymmetry correlates with poor cut value regardless of the graph topology. Its consistently safe profile makes it the recommended default when no prior knowledge about the instance family is available.

9.3 Combining heuristics does not compound their benefits

The three primary instances were tested with all heuristics active simultaneously (Figure 2, Tables 5 and 6). In no case does the combined configuration outperform the best individual heuristic for that instance. On `pm1d_100.9` it is the worst-performing configuration (-1.6% below baseline), and on `w09_100.9` and `pw09_100.9` it trails the single-heuristic leaders. The likely cause is a diversity collapse: greedy repair pins the most influential vertices to a fixed assignment, while ranked mutation simultaneously suppresses variation on those same vertices, leaving the search with very little freedom to escape local optima. Enabling all three heuristics together therefore requires careful tuning and is not advisable without instance-specific validation.

9.4 Computational cost is negligible

Figure 3 shows that all three heuristics introduce less than 3% wall-clock overhead relative to the baseline across every instance and configuration tested. Given that this variation is within normal run-to-run noise (the standard deviations in the tables confirm similar variability in raw execution times), the heuristics are effectively free in terms of computational budget. Any observed differences in solution quality are therefore attributable entirely to the operators themselves rather than to an expanded time allowance.

References

- [1] A. Wiegele, *Biq Mac Library — A Collection of Max-Cut and Quadratic 0-1 Programming Instances of Medium Size*, Technical Report, Alpen-Adria-Universität Klagenfurt, 2007. Available: <http://biqmac.aau.at/biqmaclib.html>
- [2] G. Rinaldi, *Rudy: A Graph Generator*, IASI-CNR, Rome. Available: <http://www-user.tu-chemnitz.de/~helmberg/rudy.tar.gz>
- [3] F. Liers, *Contributions to Determining Exact Ground-States of Ising Spin-Glasses and to their Physics*, Ph.D. thesis, Universität zu Köln, 2004.
- [4] F. Rendl, G. Rinaldi, and A. Wiegele, “Solving Max-Cut to Optimality by Intersecting Semidefinite and Polyhedral Relaxations,” *Mathematical Programming*, vol. 121, no. 2, pp. 307–335, 2010.
- [5] K. A. De Jong, *An Analysis of the Behavior of a Class of Genetic Adaptive Systems*, Ph.D. thesis, University of Michigan, 1975.
- [6] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*, Addison-Wesley, Reading, MA, 1989.